

COS 350 Lecture 19

review shell loop

psh2

unix: time

case study: time

exit

zombies

exec variations

waitpid



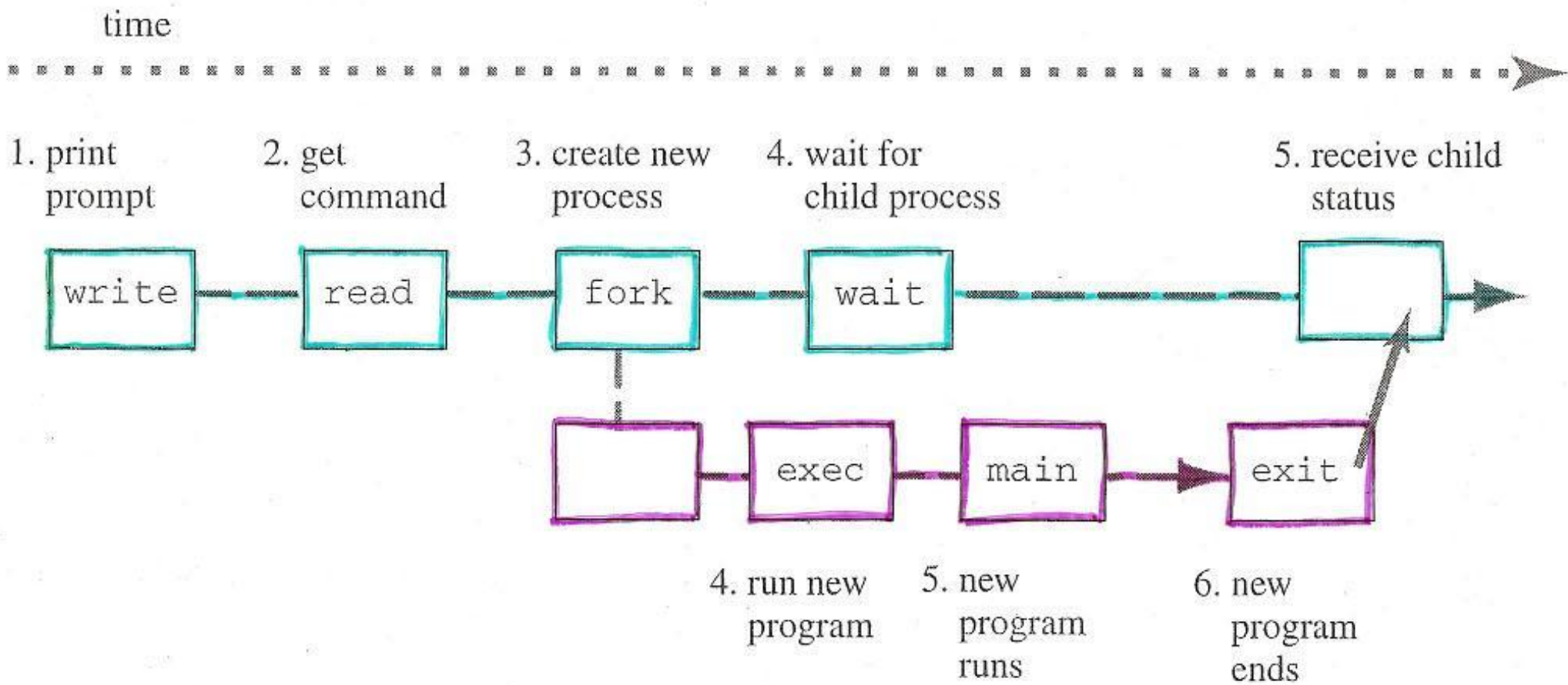
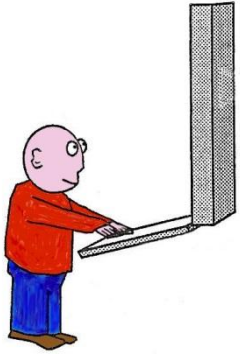


FIGURE 8.13

Shell loop with `fork()`, `exec()`, and `wait()`.



```
psh2.c
```

```
psh2
```

```
ls -l
```

```
ls -z
```

```
sleep
```

```
100
```

```
Ctrl-C
```

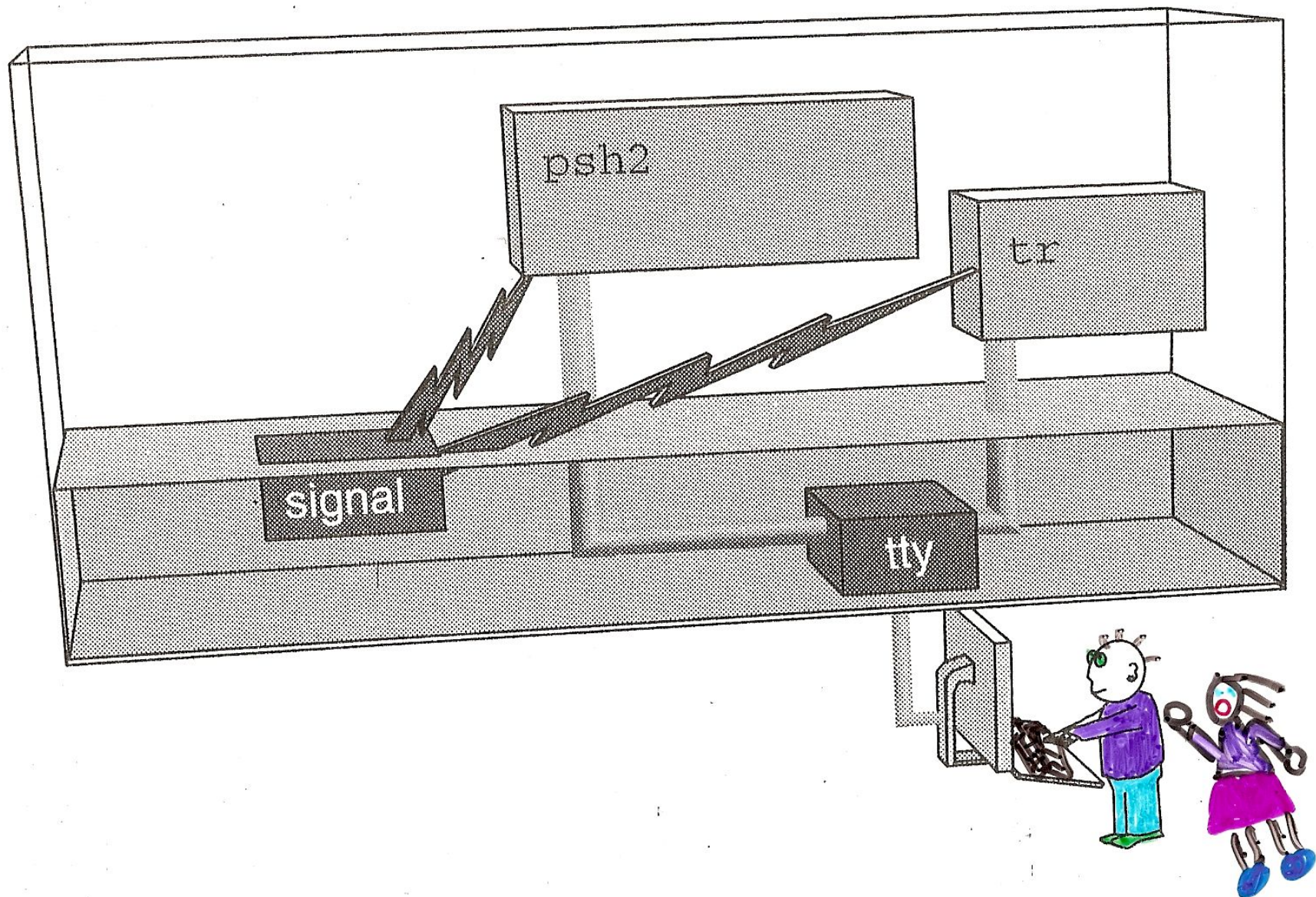


FIGURE 8.15

Keyboard signals go to all attached processes.

TIME(1)

TIME(1)

NAME

time - run programs and summarize system resource usage

SYNOPSIS

time **COMMAND** [**ARGS**]

DESCRIPTION

time runs the program **COMMAND** with any given arguments **ARGS**.
When **COMMAND** finishes, **time** displays information about resources used by **COMMAND**.

```
bash> time sort words345K -o sorted
```

```
real      0m0.507s
user      0m0.492s
sys       0m0.012s
```


NAME

gettimeofday - get time

SYNOPSIS

#include <sys/time.h>

int gettimeofday(struct timeval *tv, NULL);

DESCRIPTION

Get current Unix time (seconds since 1970).

```
struct timeval {  
    time_t      tv_sec;    /* seconds */  
    suseconds_t tv_usec;   /* microseconds */  
};
```

NAME

getrusage - get resource usage

SYNOPSIS

#include <sys/resource.h>

int getrusage(int who, struct rusage *usage);

DESCRIPTION

getrusage() returns resource usage measures for who:

RUSAGE_SELF ...

RUSAGE_CHILDREN

Return resource usage statistics for all children of the calling process that have terminated and been waited for. These statistics will include the resources used by grandchildren, and further removed descendants, if all of the intervening descendants waited on their terminated children.

The resource usages are returned in the structure pointed to by usage, which has the following form:

The resource usages are returned in the structure pointed to by usage, which has the following form:

```
struct rusage {  
    struct timeval ru_utime; /* user CPU time used */  
    struct timeval ru_stime; /* system CPU time used */  
    long ru_maxrss; /* maximum resident set size */  
    long ru_ixrss; /* integral shared memory size */  
    long ru_idrss; /* integral unshared data size */  
    long ru_isrss; /* integral unshared stack size */  
    long ru_minflt; /* page reclaims (soft page faults) */  
    long ru_majflt; /* page faults (hard page faults) */  
    long ru_nswap; /* swaps */  
    long ru_inblock; /* block input operations */  
    long ru_oublock; /* block output operations */  
    long ru_msgsnd; /* IPC messages sent */  
    long ru_msgrcv; /* IPC messages received */  
    long ru_nsignals; /* signals received */  
    long ru_nvcsw; /* voluntary context switches */  
    long ru_nivcsw; /* involuntary context switches */  
};
```

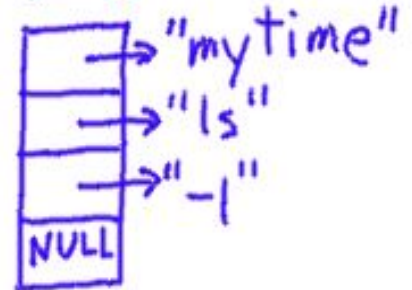


```
#include <sys/resource.h>
```

```
// mytime - run the program specified by cmd line arg  
//           and report its execution times: real, user, sys
```

```
int main(int argc, char *argv[])
```

```
{  
    int newpid, status;  
    struct timeval start, finish;
```



```
    gettimeofday(&start, NULL);    // get starting time
```

```
    if ( (newpid = fork()) == -1 ) {  
        perror("fork");  
        exit(1);  
    }
```

```
    if ( newpid == 0 ) {            // child  
        execvp(argv[1], &argv[1]);  
        perror(argv[1]);           // not expected to return  
        exit(1);  
    }
```

```
    // parent
```

```
    while (wait(&status) != newpid)    // wait for child  
        ;
```

```
    gettimeofday(&finish, NULL);    // get finishing time (real)
```

```
    double realTime = finish.tv_sec - start.tv_sec +
```

```

// parent
while (wait(&status) != newpid)    // wait for child
    ;

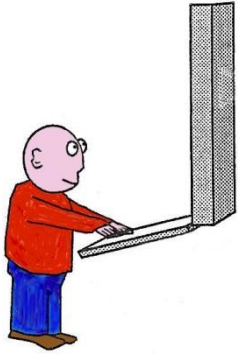
gettimeofday(&finish, NULL); // get finishing time (real)
double realTime = finish.tv_sec - start.tv_sec +
    (finish.tv_usec - start.tv_usec)/1.0e6;

struct rusage usage;
getrusage(RUSAGE_CHILDREN, &usage); // get child's times
double userTime = usage.ru_utime.tv_sec + // user time
    usage.ru_utime.tv_usec/1.0e6;
double sysTime = usage.ru_stime.tv_sec + // system time
    usage.ru_stime.tv_usec/1.0e6;

fprintf(stderr, "\n");
fprintf(stderr, "real\t%.3fs\n", realTime);
fprintf(stderr, "user\t%.3fs\n", userTime);
fprintf(stderr, "sys\t%.3fs\n", sysTime);

exit(status >> 8); // status of child (subfield encoded)
}

```



```
time sort words345K -o result  
mytime sort words345K -o result
```

NAME

exit - cause normal process termination

SYNOPSIS

```
#include <stdlib.h>
```

```
void exit(int status);
```

DESCRIPTION

The **exit()** function causes normal process termination and the value of **status** & 0377 is returned to the parent (see **wait(2)**).

All functions registered with **atexit(3)** and **on_exit(3)** are called.

All open **stdio(3)** streams are flushed and closed.

The **SIGCHLD** signal is sent to the parent.

After **exit()**, the exit status must be transmitted to the parent process.

There are three cases:

1. If the parent has set the **SIGCHLD** handler to **SIG_IGN**, the status is discarded.
2. If the parent was waiting on the child it is notified of the exit status.
3. If the parent is not waiting, the exiting process turns into a **"zombie"** process so that the parent can learn the exit status



zombiedemo.c

EXEC(3)

EXEC(3)

NAME

execl, execl, execlp, execv, execve, execvp

SYNOPSIS

#include <unistd.h>

int execl(const char *path, const char *arg, ..., NULL);

int execlp(const char *file, const char *arg, ..., NULL);

int execl(const char *path, const char *arg, ..., NULL, char *const envp[]);

int execv(const char *path, char *const argv[]);

int execvp(const char *file, char *const argv[]);

int execve(const char *path, char *const argv[], char *const envp[]);

DESCRIPTION

Each of the functions in the exec family replaces the current process image with a new process image.

Signals set to SIG_DFL and SIG_IGN in the calling process are set the same in the new process.

Signals set to be caught by the calling process are reset to SIG_DFL.

NAME

waitpid - wait for process termination

SYNOPSIS

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *status, int options);
```

DESCRIPTION

The **waitpid** function suspends execution of the current process until a child as specified by the **pid** argument has exited, or until a signal is delivered whose action is to terminate the current process or to call a signal handling function.

The value of **pid** can be one of:

- < -1** means to wait for any child process whose process group ID is equal to the absolute value of **pid**.
- 1** means to wait for any child process; this is the same behavior which **wait** exhibits.
- 0** means to wait for any child process whose process group ID is equal to that of the calling process.
- > 0** means to wait for the child whose process ID is equal to the value of **pid**.

...

The value of options is an OR of zero or more of the following constants:

WNOHANG return immediately if no child has exited.

...

The simpler `wait(&status)` call is just a special case of `waitpid(0, &status, 0)`;

RETURN VALUE

`waitpid()`: on success, returns the process ID of the child whose state has changed; if **WNOHANG** was specified and one or more child(ren) specified by pid exist, but have not yet changed state, then 0 is returned.

On error, -1 is returned.